

■ Java dla Zielonego

Kompletny przewodnik — od klas do wyjątków

Typy danych · Klasy · OOP · Wyjątki · java.time · Kolekcje · Streams · I/O

Przygotowany specjalnie do kolokwium z Programowania Obiektowego

■ Spis treści

1.	Typy danych — int, double, String i inne
2.	Klasa — budowanie własnego typu
3.	Modyfikatory dostępu — public, private, protected
4.	Konstruktor — jak tworzy obiekt
5.	this i super — słowa kluczowe
6.	static — metody i pola klasowe
7.	final — wartości niezmiennie
8.	Dziedziczenie — extends
9.	Klasa abstrakcyjna — abstract
10.	Interfejs — interface
11.	@Override — nadpisywanie metod
12.	Enum — własne typy wyliczeniowe
13.	Record — klasy danych (Java 16+)
14.	Wyjścia — wszystkie wbudowane + własne
15.	java.time — LocalDateTime, LocalDate, DateTimeFormatter
16.	String i String.format — formatowanie tekstu
17.	Tablice i ArrayList — kolekcje
18.	Map i HashMap — słowniki
19.	Streams — przetwarzanie kolekcji
20.	Pliki i ścieżki — Path, Files
21.	Wzorce dla kolokwium

1. Typy danych

Java jest językiem **statycznie typowanym** — każda zmienna musi mieć zadeklarowany typ. Typy dzielą się na dwie grupy: prymitywne (liczby, znaki, boolean) i obiektywne (String, klasy).

1.1 Typy prymitywne (wbudowane)

Typ	Opis
int	Liczba całkowita. Zakres: -2 147 483 648 do 2 147 483 647. Użycie: wiek, indeksy, godziny.
long	Liczba całkowita większa niż int. Litera: 123L (z literką L na końcu).
double	Liczba zmiennoprzecinkowa (z przecinkiem). Np. 3.14, -0.5. Uwaga: w Javie kropka, nie przecinek.
float	Mniej dokładny double. Litera: 3.14f. Rzadko używany — preferuj double.
boolean	Tylko dwie wartości: true lub false. Używany w warunkach if.
char	Pojedynczy znak: 'A', '5', ' '. Apostrofy, nie cudzysłów.
byte	Miała liczba całkowita: -128 do 127. Rzadko używana bezpośrednio.

1.2 Typy obiektywne (klasy)

Typ	Opis
String	Tekst. Zawsze w cudzysłowie: "Hello". Nie jest prymitywem — to pełna klasa.
Integer	Obiektowy odpowiednik int. Może być null. Autoboxing: int ↔ Integer automatycznie.
Double	Obiektowy odpowiednik double. Potrzebny np. w List.
List<T>	Lista obiektów. T to typ elementów, np. List, List.
Map<K, V>	Słownik klucz→wartość. Np. Map.
LocalTime	Czas (godzina:minuta:sekunda). Z pakietu java.time.
LocalDate	Data (rok-miesiąc-dzień). Z pakietu java.time.
Path	Ścieżka do pliku lub katalogu. Z pakietu java.nio.file.

1.3 Przykłady deklaracji

```
// Prymitywne
int age = 25;
double price = 9.99;
boolean isOpen = true;
char letter = 'A';

// Obiektywne
String name = "Warszawa";
Integer count = 42;           // to samo co int count = 42 (autoboxing)

// null – oznacza "brak wartości", działa tylko dla typów obiektywnych
```

```
String empty = null;           // OK
// int x = null;               // BŁĄD – prymityw nie może być null

// var – Java sama odgaduje typ (Java 10+)
var city = "Lublin";           // Java wie że to String
var num  = 5;                   // Java wie że to int
```

■ Nie używaj float — zawsze używaj double. Float ma mniejszą precyzję i sprawia problemy.

2. Klasa — budowanie własnego typu

Klasa to **przepis na obiekt**. Definiuje jakie dane obiekt przechowuje (pola) i co potrafi robić (metody). Obiekt to konkretny egzemplarz stworzony według tego przepisu.

2.1 Anatomia klasy

```
// Słowo kluczowe "class" + nazwa (z wielkiej litery, PascalCase)
public class Car {

    // === POLA — dane które obiekt przechowuje ===
    private String brand;    // marka
    private int year;        // rok produkcji
    private double price;    // cena

    // === KONSTRUKTOR — wywoływany przy tworzeniu obiektu (new Car(...)) ===
    public Car(String brand, int year, double price) {
        this.brand = brand;    // this.brand = pole klasy, brand = parametr
        this.year = year;
        this.price = price;
    }

    // === METODY — co obiekt potrafi robić ===
    public String getBrand() {    // getter — zwraca wartość pola
        return brand;
    }

    public void setPrice(double price) {    // setter — zmienia wartość pola
        this.price = price;
    }

    public boolean isNew() {    // metoda obliczająca coś
        return year >= 2020;
    }

    @Override
    public String toString() {    // konwersja na tekst
        return brand + " (" + year + ")";
    }
}

// === UŻYCIE — tworzenie obiektu ===
Car car = new Car("Toyota", 2022, 85000.0);
System.out.println(car.getBrand());    // "Toyota"
System.out.println(car.isNew());    // true
System.out.println(car);    // "Toyota (2022)" — wywołuje toString()
```

2.2 Różnica między klasą a obiektem

Klasa	Przepis, schemat. Istnieje jeden w kodzie. Np. klasa Car.
Obiekt	Konkretny egzemplarz. Może być wiele. Np. car1, car2 — oba są Car.
new	Słowo kluczowe tworzące obiekt. Car car = new Car(...)

`null`

Zmienna obiektowa niezainicjalizowana / pusta. Wywołanie metody na null →
NullPointerException!

3. Modyfikatory dostępu — public, private, protected

Modyfikatory kontrolują kto może czytać/zmieniać pole lub wywoływać metodę. To mechanizm ochrony danych — ukrywamy to czego nie chcemy udostępnić na zewnątrz.

Modyfikator	Widoczny dla	Kiedy używać
public	Wszystkich — innych klas, podklas, zewnętrznych	Metody API, getter/setter, konstruktory
private	Tylko tej samej klasy	Pola klasy (dane wewnętrzne), metody pomocnicze
protected	Tej klasy + podklas (klas dziedziczonych)	Pola w klasie abstrakcyjnej, które podklasy muszą czytać
(brak)	Tylko klas w tym samym pakiecie (folderze)	Rzadko używane celowo

```
public class BankAccount {
    private double balance;    // prywatne – nikt z zewnątrz nie zmieni bezpośrednio

    public double getBalance() {    // publiczny getter – można czytać
        return balance;
    }

    public void deposit(double amount) {    // publiczna metoda – można wpłacać
        if (amount > 0) balance += amount;
    }

    private void log(String msg) {    // prywatna metoda pomocnicza
        System.out.println("[LOG] " + msg);
    }
}

// Z zewnątrz:
BankAccount acc = new BankAccount();
acc.deposit(100);
System.out.println(acc.getBalance()); // OK
// acc.balance = 999999;                // BŁĄD KOMPILACJI – balance jest private
```

■ Zasada ogólna: pola klasy zawsze private lub protected. Dostęp przez getter (metody publiczne). To się nazywa enkapsulacja.

4. Konstruktor — jak tworzy się obiekt

Konstruktor to specjalna metoda wywoływana **automatycznie** przy tworzeniu obiektu (new). Ma taki sam **nazwę** jak klasa i nie ma typu zwracanego (nawet void).

```
public class City {
    private String name;
    private int timezone;
    private double longitude;

    // Konstruktor z parametrami
    public City(String name, int timezone, double longitude) {
        this.name = name;           // this = ten obiekt który właśnie tworzymy
        this.timezone = timezone;
        this.longitude = longitude;
    }

    // Konstruktor bezparametrowy (domyślny)
    // Jeśli nie napiszesz żadnego – Java sama doda pusty
    // Jeśli napiszesz jakikolwiek konstruktor – Java już tego NIE robi
    public City() {
        this.name = "Nieznane";
        this.timezone = 0;
        this.longitude = 0.0;
    }
}

// Użycie:
City warsaw = new City("Warszawa", 2, 21.01); // wywołuje 1. konstruktor
City empty = new City();                      // wywołuje 2. konstruktor
```

4.1 Kiedy potrzebujesz kilku konstruktorów?

Gdy chcesz tworzyć obiekt na różne sposoby — z pełnymi danymi lub z częściowymi. Java rozróżnia konstruktory po liczbie i typach parametrów (przeciążenie).

■ ■ Jeśli napiszesz konstruktor z parametrami, Java nie doda już domyślnego pustego konstruktora. Jeśli potrzebujesz `new City()` — musisz go dopisać ręcznie.

5. this i super — słowa kluczowe

5.1 this — odniesienie do bieżącego obiektu

<code>this.pole</code>	Odniesienie do pola klasy. Rozróżnia pole od parametru o tej samej nazwie.
<code>this(args)</code>	Wywołanie innego konstruktora tej samej klasy. MUSI być pierwszą linią konstruktora.
<code>this</code>	Cały bieżący obiekt jako referencja. Np. do przekazania siebie jako argumentu.

```
public class Clock {
    protected int hour, minute, second;

    public Clock(int hour, int minute, int second) {
        this.hour = hour;    // this.hour = pole, hour = parametr
        this.minute = minute;
        this.second = second;
    }

    public Clock() {
        this(0, 0, 0);    // wywołuje konstruktor powyżej z wartościami 0,0,0
    }
}
```

5.2 super — odniesienie do klasy nadrzędnej

<code>super(args)</code>	Wywołanie konstruktora klasy nadrzędnej. MUSI być pierwszą linią konstruktora podklasy.
<code>super.metoda()</code>	Wywołanie metody z klasy nadrzędnej (gdy podklasa ją nadpisała).

```
public abstract class Clock {
    protected int hour;

    public Clock(int hour) {
        this.hour = hour;
    }

    public String toString() {
        return String.format("%02d:00:00", hour);
    }
}

public class DigitalClock extends Clock {

    public DigitalClock(int hour) {
        super(hour);    // wywołuje konstruktor Clock(hour)
    }

    @Override
    public String toString() {
        // super.toString() zwróci "hh:00:00" z klasy Clock
        return "Digital: " + super.toString();
    }
}
```

■ Zasada: `super()` w konstruktorze podklasy ZAWSZE jako pierwsza linia, inaczej błąd kompilacji.

6. static — metody i pola klasowe

Normalnie metody i pola należą do **konkretnego obiektu**. Słowo `static` sprawia że należą do **klasy jako całości** — nie potrzebujesz tworzyć obiektu żeby ich użyć.

```
public class MathHelper {

    // Pole statyczne – wspólne dla wszystkich, jedno w pamięci
    public static final double PI = 3.14159;

    // Metoda statyczna – wywoływana przez nazwę klasy, nie obiektu
    public static double circleArea(double radius) {
        return PI * radius * radius;
    }
}

// Użycie:
double area = MathHelper.circleArea(5.0); // bez new MathHelper()
double pi    = MathHelper.PI;

// Znany przykład: metody w klasie Math
double root = Math.sqrt(16); // Math.sqrt jest static
double max  = Math.max(3, 7); // Math.max jest static
```

6.1 Statyczna fabryka `fromCsv` — wzorzec z kolokwium

Na kolokwium często prosi się o statyczną metodę, która wczytuje plik i zwraca obiekt. To wzorzec 'factory method' — alternatywa dla konstruktora.

```
public class City {
    private String name;
    private int timezone;

    // Prywatny konstruktor – nikt nie tworzy City przez new City(...)
    private City(String name, int timezone) {
        this.name = name;
        this.timezone = timezone;
    }

    // Publiczna statyczna metoda fabryczna
    public static City parseLine(String line) {
        String[] cols = line.split(",");
        String name    = cols[0].trim();
        int timezone   = Integer.parseInt(cols[1].trim());
        return new City(name, timezone); // wywołuje prywatny konstruktor
    }
}

// Użycie:
City city = City.parseLine("Warszawa,2,52.23,21.01");
```

7. final — wartości niezmiennie

final na polu	Pole można ustawić tylko raz (w konstruktorze lub przy deklaracji). Potem jest tylko do odczytu.
final na metodzie	Metoda nie może być nadpisana (@Override) w podklasie.
final na klasie	Klasa nie może być rozszerzana (extends). Np. klasa String jest final.
final na zmiennej lokalnej	Zmienna lokalna nie może być zmieniona po pierwszym przypisaniu.

```
public class City {  
    private final String name;        // final – ustawiane w konstruktorze, potem stałe  
    private final double longitude;  
  
    public City(String name, double longitude) {  
        this.name = name;  
        this.longitude = longitude;  
    }  
  
    // name i longitude już nigdy nie można zmienić  
}
```

■ Używaj final na polach gdy obiekt nie powinien zmieniać swoich danych po stworzeniu. To czyni kod bardziej przewidywalnym.

8. Dziedziczenie — extends

Dziedziczenie pozwala jednej klasie **przejmować** wszystkie pola i metody innej klasy. Podklasa dostaje wszystko co ma nadklasa — i **może** dodać lub nadpisać.

```
// Klasa bazowa (nadklasa)
public class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public void breathe() {
        System.out.println(name + " oddycha");
    }

    public String sound() {
        return "...";
    }
}

// Klasa pochodna (podklasa) – dziedziczy po Animal
public class Dog extends Animal {

    public Dog(String name) {
        super(name);    // wywołaj konstruktor Animal
    }

    @Override
    public String sound() {    // nadpisujemy metodę
        return "Hau!";
    }

    public void fetch() {    // nowa metoda tylko w Dog
        System.out.println(name + " aportuje");
    }
}

// Użycie:
Dog dog = new Dog("Reksio");
dog.breathe();    // "Reksio oddycha" – z klasy Animal (odziedziczone)
dog.sound();      // "Hau!" – z klasy Dog (nadpisane)
dog.fetch();      // "Reksio aportuje" – tylko w Dog

// Polimorfizm – Dog może być traktowany jako Animal
Animal a = new Dog("Burek");
a.breathe();      // działa
a.sound();        // "Hau!" – Java wywoła metodę Dog, nie Animal
// a.fetch();     // BŁĄD – typ zmiennej to Animal, który nie zna fetch()
```

8.1 Hierarchia klas w kolokwium 2024

Clock (abstract)

setTime() · setCurrentTime() · toString() · setCity()

DigitalClock (krok 2) Mode enum
toString() — 12h/24h

AnalogClock (krok 7) list of ClockHand
toSvg(Path)

■ Każda podklasa jest też obiektem nadklasy. DigitalClock IS-A Clock. Dlatego możesz trzymać DigitalClock w zmiennej typu Clock.

9. Klasa abstrakcyjna — abstract

Klasa abstrakcyjna to klasa której **nie można tworzyć bezpośrednio** (`new Clock()` → błąd). Służy jako baza dla podklas. Może mieć metody abstrakcyjne — bez ciała, które podklasy **MUSZĄ** zaimplementować.

```
// Klasa abstrakcyjna – szablon dla podklas
public abstract class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    // Metoda abstrakcyjna – BRAK ciała ({})
    // Każda podklasa musi ją zaimplementować
    public abstract double area();

    // Zwykła metoda – podklasy dziedziczą gotową implementację
    public void describe() {
        System.out.println("Kształt " + color + ", pole: " + area());
    }
}

// Konkretna podklasa – MUSI zaimplementować area()
public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double area() { // implementacja metody abstrakcyjnej
        return Math.PI * radius * radius;
    }
}

// Użycie:
// Shape s = new Shape("red"); // BŁĄD – abstract class
Circle c = new Circle("red", 5.0);
c.describe(); // działa – describe() jest w Shape, area() jest w Circle
```

9.1 Kiedy używać klasy abstrakcyjnej?

Wspólne dane	Nadklasa trzyma wspólne pola (hour, minute, second w Clock).
Wspólna logika	Często metod jest ta sama dla wszystkich podklas (setTime, setCurrentTime).
Wymagana implementacja	Każda podklasa musi zaimplementować coś inaczej (toString w DigitalClock vs AnalogClock).
Nie ma sensu sam w sobie	Nie chcesz żeby ktoś zrobił <code>new Clock()</code> — zegar musi być cyfrowy lub analogowy.

10. Interfejs — interface

Interfejs to **kontrakt** — lista metod które klasa MUSI zaimplementować. Nie ma pól (oprócz stałych), nie ma konstruktora, wszystkie metody są publiczne i abstrakcyjne.

```
// Interfejs – tylko deklaracje metod
public interface Drawable {
    String toSvg();           // każda klasa implementująca musi to zrobić
    void draw();
}

// Klasa implementuje interfejs – słowo "implements"
public class Circle implements Drawable {
    @Override
    public String toSvg() {
        return "<circle .../>";
    }

    @Override
    public void draw() {
        System.out.println("Rysuj koło");
    }
}

// Interfejs funkcyjny (@FunctionalInterface) – dokładnie 1 metoda abstrakcyjna
// Można przekazywać jako argument (lambda lub referencja do metody)
@FunctionalInterface
interface CsvFactory<T> {
    T create(Path path) throws IOException;
}

// Użycie z referencją do metody (kolokwium 2022):
addAll(City::parseLine, Path.of("strefy.csv"));
```

10.1 Różnica: interfejs vs klasa abstrakcyjna

Klasa abstrakcyjna	Może mieć pola, konstruktory, gotowe metody. Dziedziczenie: extends (tylko jedna!).
Interfejs	Brak pól (tylko stałe), brak konstruktora, same deklaracje. Implementacja: implements (można wiele!).

■ W kolokwium 2024: Clock jest klasą abstrakcyjną (ma pola hour/minute/second, gotowe metody). ClockHand jest klasą abstrakcyjną (ma metody abstrakcyjne toSvg, setTime).

11. @Override — nadpisywanie metod

@Override mówi Javie: 'ta metoda nadpisuje metodę z klasy nadrzędnej'. Technicznie adnotacja jest opcjonalna, ale **zawsze ją pisz** — kompilator sprawdzi czy naprawdę nadpisujesz (wyłapie literówki w nazwie metody).

```
public class DigitalClock extends Clock {

    // @Override – nadpisujemy toString() z klasy Clock
    @Override
    public String toString() {
        // super.toString() wywołuje toString() z Clock
        return "Digital: " + super.toString();
    }
}

// BEZ @Override – śladnego błędu przy literówce:
public class BuggyClass extends Clock {
    public String toString() { // literówka! małe 's'
        return "???"; // Java myśli że to NOWA metoda, nie nadpisanie
    }
}

// Z @Override – kompilator wyłapie:
public class SafeClass extends Clock {
    @Override
    public String toString() { // BŁĄD KOMPILACJI – "nie ma takiej metody w Clock"
        return "???";
    }
}
```

12. Enum — własne typy wyliczeniowe

Enum to specjalny typ z **predefiniowanymi**, **zamkniętymi** listami **wartości**. Zamiast `int mode = 12` lub `String mode = 'TWELVE'` — tworzysz własny typ.

```
// Definicja enum — wewnątrz klasy lub jako osobny plik
public enum Season {
    SPRING, SUMMER, AUTUMN, WINTER
}

// Enum wewnątrz klasy (jak w kolokwium 2024)
public class DigitalClock extends Clock {

    public enum Mode {
        TWELVE_HOUR,
        TWENTY_FOUR_HOUR
    }

    private final Mode mode;

    public DigitalClock(Mode mode) {
        this.mode = mode;
    }
}

// Użycie enum:
DigitalClock.Mode m = DigitalClock.Mode.TWELVE_HOUR;

// Porównanie enum — zawsze ==, nie .equals()
if (mode == Mode.TWELVE_HOUR) { ... }

// Switch na enum
switch (mode) {
    case TWELVE_HOUR      -> System.out.println("12h");
    case TWENTY_FOUR_HOUR-> System.out.println("24h");
}

// Enum może mieć pola i metody
public enum Resource {
    COAL(true), FISH(false), WOOD(true);    // wartości z parametrami

    private final boolean inland;
    Resource(boolean inland) { this.inland = inland; }
    public boolean isInland() { return inland; }
}

// Konwersja enum ↔ String
String s = Season.SPRING.name();           // "SPRING"
Season sp = Season.valueOf("SPRING");      // Season.SPRING
```

13. Record — klasy danych (Java 16+)

Record to skrót dla klas które tylko przechowują dane. Kompilator sam generuje konstruktor, getters, equals(), hashCode(), toString().

```
// Normalnie musiałyby napisać:
public class Point {
    private final double x;
    private final double y;
    public Point(double x, double y) { this.x = x; this.y = y; }
    public double x() { return x; }
    public double y() { return y; }
    // + equals, hashCode, toString ...
}

// Z record – to samo ale w jednej linii!
public record Point(double x, double y) { }

// Użycie – UWAGA: getters bez "get" w nazwie (x(), nie getX())
Point p = new Point(3.0, 4.0);
double x = p.x();    // nie p.getX()!
double y = p.y();

// Można dodać metody do record:
public record Point(double x, double y) {
    public double distanceTo(Point other) {
        return Math.sqrt(Math.pow(x - other.x, 2) + Math.pow(y - other.y, 2));
    }
}
```

■ Record jest zawsze immutable (niezmienialny) — nie można zmienić x ani y po stworzeniu. To jego cecha, nie ograniczenie.

14. Wyjątki — błądy w czasie działania programu

Wyjątek (Exception) to obiekt który Java tworzy gdy coś idzie nie tak. Możesz go przechwycić (try-catch) albo rzucić dalej (throws). Istnieją dwie kategorie: checked (kompilator wymaga obsługi) i unchecked (nie wymaga).

14.1 Hierarchia wyjątków

```
Throwable
├── Error (błądy JVM – nie łap: OutOfMemoryError, StackOverflowError)
├── Exception
│   ├── IOException ← CHECKED – musisz obsługiwać (throws lub try-catch)
│   │   ├── FileNotFoundException
│   │   └── RuntimeException ← UNCHECKED – obsługa opcjonalna
│   │       ├── NullPointerException
│   │       ├── IllegalArgumentException
│   │       ├── IndexOutOfBoundsException
│   │       ├── NumberFormatException
│   │       ├── ClassCastException
│   │       ├── ArithmeticException
│   │       └── (inne checked exceptions)
```

14.2 Wbudowane wyjątki — kompletna lista

Wyjątek	Kiedy i jak
NullPointerException	Wywołanie metody lub dostęp do pola na zmiennej null. Przykład: String s = null; s.length(); → NPE
IllegalArgumentException	Przekazano nieprawidłowy argument do metody. Użycie na kolokwium: setTime(-1, 0, 0) → godzina poza zakresem
IllegalStateException	Obiekt jest w nieodpowiednim stanie do wywołania metody.
IndexOutOfBoundsException	Indeks poza granicami tablicy lub listy. Przykład: list.get(10) gdy lista ma 5 elementów
ArrayIndexOutOfBoundsException	Indeks poza granicami tablicy. Podtyp IndexOutOfBoundsException.
NumberFormatException	Konwersja stringa na liczbę gdy string nie jest liczbą. Przykład: Integer.parseInt("abc") lub Double.parseDouble("1,5")
ClassCastException	Rzutowanie obiektu na nieodpowiedni typ. Przykład: Object o = new Dog(); String s = (String) o;
ArithmeticException	Błąd arytmetyczny. Np. dzielenie int przez 0 (5/0).
StackOverflowError	Nieskończona rekurencja. Metoda wywołuje samą siebie bez końca.
IOException	Błąd wejścia/wyjścia. Np. brak pliku, brak uprawnień. CHECKED — wymagane try-catch.
FileNotFoundException	Plik nie istnieje. Podtyp IOException. CHECKED.
UnsupportedOperationException	Operacja nie jest obsługiwana. Np. modyfikacja listy zwróconej przez List.of().

14.3 Jak rzucać i łapać

```

// RZUCANIE – throw new Wyj tekException("opis")
public void setTime(int hour, int minute, int second) {
    if (hour < 0 || hour > 23) {
        throw new IllegalArgumentException(
            "Godzina musi by  0-23, podano: " + hour
        );
    }
    // ...
}

//  APANIE – try { ... } catch (Typ e) { ... }
try {
    clock.setTime(25, 0, 0);
} catch (IllegalArgumentException e) {
    System.out.println("B  d: " + e.getMessage());    // "Godzina musi by  0-23, podano: 25"
}

//  APANIE WIELU wyj tk w
try {
    Path path = Path.of("plik.csv");
    List<String> lines = Files.readAllLines(path);
} catch (FileNotFoundException e) {
    System.out.println("Nie znaleziono pliku: " + e.getMessage());
} catch (IOException e) {
    System.out.println("B  d odczytu: " + e.getMessage());
}

// DEKLAROWANIE – throws (dla checked exceptions)
public static City fromCsv(Path path) throws IOException {
    // tu mog  wyrzuci  IOException bez try-catch
    List<String> lines = Files.readAllLines(path);
    // ...
}

```

14.4 W asny wyj tek

```

// Checked – kompilator wymusza try-catch
class CityNotFoundException extends Exception {
    public CityNotFoundException(String cityName) {
        super(cityName);    // getMessage() zwr ci cityName
    }
}

// Unchecked – nie wymaga try-catch
class CityNotOnLandException extends RuntimeException {
    public CityNotOnLandException(String cityName) {
        super(cityName);
    }
}

// U ycie w asnego wyj tku:
// Throws checked:
public City findCity(String name) throws CityNotFoundException {
    City c = cities.get(name);
}

```

```

        if (c == null) throw new CityNotFoundException(name);
        return c;
    }

    // Throws unchecked:
    public void addCity(City city) {
        if (!land.inside(city.center())) {
            throw new CityNotOnLandException(city.name()); // bez throws w sygnaturze
        }
    }

    // W teście JUnit:
    RuntimeException ex = assertThrows(RuntimeException.class, () -> {
        land.addCity(cityOnWater);
    });
    assertEquals("Atlantyda", ex.getMessage());

```

15. java.time — operacje na czasie i dacie

Pakiet `java.time` (Java 8+) zawiera klasy do pracy z czasem i datą. Wszystkie są `immutable` — operacje zwracają nowy obiekt, nie modyfikują istniejącego.

15.1 LocalDateTime — czas (bez daty)

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

// Tworzenie
LocalTime now = LocalDateTime.now();           // bieżący czas systemowy
LocalTime t    = LocalDateTime.of(13, 30, 45); // 13:30:45
LocalTime t2   = LocalDateTime.of(0, 0, 0);    // północ

// Odczyt składowych
int h = t.getHour();      // 13
int m = t.getMinute();   // 30
int s = t.getSecond();    // 45
int n = t.getNano();      // nanosekundy (zwykle 0)

// Operacje — każda zwraca NOWY obiekt (immutable!)
LocalTime later  = t.plusHours(2);             // 15:30:45
LocalTime before = t.minusMinutes(15);         // 13:15:45
LocalTime shifted = t.plusSeconds(3661);       // +1h 1min 1s

// Porównanie
boolean before2 = t.isBefore(LocalTime.NOON); // czy przed 12:00?
boolean after2  = t.isAfter(LocalTime.of(8,0)); // czy po 8:00?

// Formatowanie
String s24 = t.format(DateTimeFormatter.ofPattern("HH:mm:ss")); // "13:30:45"
String s12 = t.format(DateTimeFormatter.ofPattern("hh:mm:ss a")); // "01:30:45 PM"

// Stałe
LocalTime noon      = LocalDateTime.NOON;      // 12:00
LocalTime midnight  = LocalDateTime.MIDNIGHT;  // 00:00
LocalTime max       = LocalDateTime.MAX;       // 23:59:59.999...
```

15.2 LocalDate — data (bez czasu)

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

// Tworzenie
LocalDate today = LocalDate.now();
LocalDate d     = LocalDate.of(2021, 1, 5);    // 5 stycznia 2021
LocalDate d2    = LocalDate.parse("2021-01-05"); // z napisu (format ISO)

// Parsowanie własnego formatu
LocalDate d3 = LocalDate.parse("5/1/21",
    DateTimeFormatter.ofPattern("d/M/yy"));

// Parsowanie formatu US: "1/5/21" = January 5th 2021
LocalDate d4 = LocalDate.parse("1/5/21",
```

```

        DateTimeFormatter.ofPattern("M/d/yy"));

// Odczyt
int year  = d.getYear();           // 2021
int month = d.getMonthValue();    // 1 (styczeń)
int day   = d.getDayOfMonth();    // 5

// Operacje
LocalDate tomorrow = d.plusDays(1);
LocalDate nextMonth = d.plusMonths(1);
LocalDate nextYear  = d.plusYears(1);

// Porównanie zakresów (wzorzec z kolokwium 2021)
boolean inRange = !d.isBefore(startDate) && !d.isAfter(endDate);

// Iteracja po datach (Java 9+)
startDate.datesUntil(endDate.plusDays(1)).forEach(date -> {
    System.out.println(date);
});

// Formatowanie
String s = d.format(DateTimeFormatter.ofPattern("d.MM.yy")); // "5.01.21"

```

15.3 DateTimeFormatter — wzorce formatu

Wzorzec	Znaczenie
yyyy	Rok 4-cyfrowy: 2024
yy	Rok 2-cyfrowy: 24
MM	Miesiąc 2-cyfrowy: 01, 12
M	Miesiąc bez zera: 1, 12
dd	Dzień 2-cyfrowy: 05
d	Dzień bez zera: 5
HH	Godzina 24h, 2-cyfrowa: 00–23
H	Godzina 24h, bez zera: 0–23
hh	Godzina 12h, 2-cyfrowa: 01–12
h	Godzina 12h, bez zera: 1–12
mm	Minuta 2-cyfrowa: 00–59
ss	Sekunda 2-cyfrowa: 00–59
a	AM lub PM

16. String i formatowanie tekstu

16.1 Najważniejsze metody String

Metoda	Opis
<code>s.trim()</code>	Usuwa białe znaki z początku i końca: " abc " → "abc"
<code>s.split(sep)</code>	Dzieli string na tablicę po separatorze. <code>split(",")</code> lub <code>split(";", -1)</code>
<code>s.startsWith(pre)</code>	Czy string zaczyna się od podanego prefixu. Zwraca boolean.
<code>s.endsWith(suf)</code>	Czy string kończy się podanym sufiksem. Zwraca boolean.
<code>s.contains(sub)</code>	Czy string zawiera podciąg. Zwraca boolean.
<code>s.replace(a, b)</code>	Zamienia wszystkie wystąpienia a na b.
<code>s.toLowerCase()</code>	Zamienia na małe litery.
<code>s.toUpperCase()</code>	Zamienia na wielkie litery.
<code>s.length()</code>	Długość stringa.
<code>s.charAt(i)</code>	Znak na pozycji i.
<code>s.substring(i)</code>	Podciąg od pozycji i do końca.
<code>s.substring(i, j)</code>	Podciąg od i do j (j nie wliczone).
<code>s.equals(other)</code>	Porównanie treści stringów. NIGDY nie używaj <code>==</code> dla stringów!
<code>s.isEmpty()</code>	Czy string jest pusty ("").
<code>String.join(sep, list)</code>	Łączy listę w string ze separatorem.

16.2 String.format — formatowanie

```
// Wzorzec: %[flagi][szerokość]typ
// Typy: %d = int, %f = double, %s = String, %c = char, %n = nowa linia

// Liczba całkowita
String.format("%d", 5)           // "5"
String.format("%5d", 5)          // "    5" (5 szerokości, wyrównane prawo)
String.format("%-5d", 5)         // "5    " (wyrównane lewo)
String.format("%05d", 5)         // "00005" (uzupełnione zerami)
String.format("%02d", 5)         // "05" (2 cyfry, zera wiodące)

// Double
String.format("%.2f", 3.14159)   // "3.14" (2 miejsca po przecinku)
String.format("%8.2f", 3.14)     // "    3.14" (8 szerokości)

// String
String.format("%-10s", "abc")    // "abc      " (wyrównane lewo, 10 szerokości)

// Łączenie
String.format("%02d:%02d:%02d", 1, 5, 9) // "01:05:09"

// Nowa linia – użyj %n (systemowa) lub \n
```

```
String.format("Linia 1\nLinia 2")
```

```
// printf – to samo co format ale od razu wypisuje  
System.out.printf("Godzina: %02d:%02d\n", hour, minute);
```

■ Do godzin/minut/sekund zawsze %02d. Do normalnych liczb %d. Do liczb z miejscami po przecinku %.Nf gdzie N to ilo■■ miejsc.

17. Tablice i ArrayList — kolekcje elementów

17.1 Tablica — stały rozmiar

```
// Deklaracja i inicjalizacja
int[] numbers = new int[5];           // tablica 5 intów, wypełniona zerami
String[] names = new String[3];       // tablica 3 stringów, wypełniona null
double[] prices = {1.5, 2.3, 9.99};  // tablica z wartościami

// Dostęp przez indeks (od 0!)
numbers[0] = 42;
System.out.println(prices[1]);      // 2.3

// Długość
int len = numbers.length;           // uwaga: length bez (), nie length()!

// Pętla
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}

// Pętla for-each (elegancka, ale bez indeksu)
for (double price : prices) {
    System.out.println(price);
}

// Konwersja tablica → lista
List<Double> list = Arrays.asList(prices);    // stała długość
List<Double> list2 = new ArrayList<>(Arrays.asList(prices)); // modyfikowalna
```

17.2 ArrayList — dynamiczna lista

```
import java.util.ArrayList;
import java.util.List;

// Tworzenie
List<String> cities = new ArrayList<>(); // pusta lista
List<String> cities2 = new ArrayList<>(List.of("Warszawa", "Kraków"));

// Dodawanie
cities.add("Gdańsk");
cities.add(0, "Lublin"); // dodaj na indeksie 0

// Dostęp
String first = cities.get(0);
int size = cities.size(); // .size() a nie .length!

// Usuwanie
cities.remove("Gdańsk"); // usuw po wartości
cities.remove(0);        // usuw po indeksie

// Sprawdzenie
boolean has = cities.contains("Lublin"); // true/false
boolean empty = cities.isEmpty();
```

```
// Pętla
for (String city : cities) {
    System.out.println(city);
}

// Niemutowalne listy (nie można dodawać/usuwać!)
List<String> fixed = List.of("A", "B", "C"); // Java 9+
```

18. Map i HashMap — słowniki klucz→warto

Map przechowuje pary klucz→warto. Każdy klucz jest unikalny. HashMap to najczęstsza implementacja (szybka, nieuporządkowana). LinkedHashMap zachowuje kolejno wstawiania. TreeMap sortuje po kluczu.

```
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

// Tworzenie
Map<String, City> cityMap = new HashMap<>();
Map<String, City> ordered = new LinkedHashMap<>(); // zachowuje kolejno

// Dodawanie
cityMap.put("Warszawa", warsawCity);
cityMap.put("Kraków", krakowCity);

// Odczyt
City city = cityMap.get("Warszawa"); // null jeśli nie ma klucza
City city2 = cityMap.getOrDefault("X", defaultCity); // bezpieczny

// Sprawdzenie
boolean exists = cityMap.containsKey("Lublin");
boolean hasCity = cityMap.containsValue(someCity);
int size = cityMap.size();

// Usuwanie
cityMap.remove("Kraków");

// Iteracja po wszystkich parach
for (Map.Entry<String, City> entry : cityMap.entrySet()) {
    String key = entry.getKey();
    City value = entry.getValue();
    System.out.println(key + " → " + value);
}

// Iteracja tylko po kluczach
for (String key : cityMap.keySet()) { ... }

// Iteracja tylko po wartościach
for (City c : cityMap.values()) { ... }

// Wzorzec z kolokwium – parseFile zwraca Map<String, City>
public static Map<String, City> parseFile(Path path) throws IOException {
    Map<String, City> result = new LinkedHashMap<>();
    List<String> lines = Files.readAllLines(path);
    for (int i = 1; i < lines.size(); i++) { // pomiń nagłówki
        City city = parseLine(lines.get(i));
        result.put(city.getName(), city);
    }
    return result;
}
```

19. Streams — przetwarzanie kolekcji

Stream to potok operacji na kolekcji. Zamiast pisać wiele metod. Stream nie modyfikuje oryginalnej listy — tworzy nową.

```
import java.util.stream.*;
import java.util.Comparator;

List<String> cities = List.of("Warszawa", "Kraków", "Lublin", "Gdańsk");

// Filtrowanie – zostają tylko elementy spełniające warunek
List<String> longNames = cities.stream()
    .filter(c -> c.length() > 6)
    .collect(Collectors.toList());    // ["Warszawa", "Kraków"]

// Mapowanie – transformacja każdego elementu
List<Integer> lengths = cities.stream()
    .map(c -> c.length())             // lub .map(String::length)
    .collect(Collectors.toList());    // [8, 6, 6, 6]

// Sortowanie
List<String> sorted = cities.stream()
    .sorted(Comparator.comparing(c -> c))
    .collect(Collectors.toList());

// Suma / average (mapToInt/mapToDouble)
int totalLen = cities.stream()
    .mapToInt(String::length)
    .sum();

double avgLen = cities.stream()
    .mapToDouble(String::length)
    .average()
    .orElse(0.0);

// Szukanie pierwszego pasującego
Optional<String> found = cities.stream()
    .filter(c -> c.startsWith("L"))
    .findFirst();

String result = found.orElse("brak");    // "Lublin" lub "brak"
boolean has    = found.isPresent();      // true

// Wzorzec z kolokwium – getProducts (2022)
List<Product> matching = products.stream()
    .filter(p -> p.getName().startsWith(prefix))
    .collect(Collectors.toList());

// Stream.of – strumień z podanych wartości (dla @MethodSource w JUnit)
Stream<Arguments> cases() {
    return Stream.of(
        Arguments.of("A", true),
        Arguments.of("B", false)
    );
}
```

```
}
```

19.1 Mapa skrótów — stream w jednej linii

Metoda	Co robi
<code>.filter(x -> warunek)</code>	Zostają elementy dla których warunek == true
<code>.map(x -> f(x))</code>	Zamienia każdy element — zmienia typ lub wartość
<code>.sorted(Comparator)</code>	Sortuje. Bez argumentu — naturalny porządek
<code>.distinct()</code>	Usuwa duplikaty
<code>.limit(n)</code>	Bierze tylko n pierwszych elementów
<code>.forEach(x -> ...)</code>	Wykonuje akcję dla każdego — KOŃCZY stream, nic nie zwraca
<code>.collect(toList())</code>	Zbiera wyniki do List
<code>.count()</code>	Liczy elementy
<code>.sum() / .average()</code>	Tylko po mapToInt/mapToDouble. Suma / średnia.
<code>.findFirst()</code>	Pierwszy element jako Optional
<code>.anyMatch(x -> war)</code>	Czy jakikolwiek element spełnia warunek
<code>.allMatch(x -> war)</code>	Czy wszystkie elementy spełniają warunek

20. Pliki i ścieżki — Path i Files

Java NIO (pakiet `java.nio.file`) to nowoczesny sposób pracy z plikami. Path to ścieżka, Files to zestaw metod do operacji na plikach.

```
import java.nio.file.*;
import java.io.*;

// === PATH – ścieżka do pliku / katalogu ===
Path p1 = Path.of("data/strefy.csv");           // względną
Path p2 = Path.of("/home/user/data.csv");       // bezwzględną
Path p3 = Path.of("data").resolve("plik.csv"); // ścieżka: "data/plik.csv"

// Sprawdzanie
boolean exists = Files.exists(p1);
boolean readable = Files.isReadable(p1);
boolean isDir = Files.isDirectory(p1);

// === ODCZYT ===
// Wszystkie linie naraz (małe pliki)
List<String> lines = Files.readAllLines(p1);

// Czytaj plik jako String
String content = Files.readString(p1);

// Linia po linii (Scanner – dla dużych plików)
try (Scanner sc = new Scanner(p1)) {
    while (sc.hasNextLine()) {
        String line = sc.nextLine();
    }
} // try-with-resources – plik zamknięty automatycznie!

// === ZAPIS ===
// Zapisz String do pliku
Files.writeString(Path.of("output.svg"), svgContent);

// Zapisz wiele linii
Files.write(Path.of("output.txt"), List.of("linia1", "linia2"));

// PrintWriter – zapis z formatowaniem
try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(Path.of("out.txt")))) {
    pw.printf("%02d:%02d\t%d%n", hour, minute, value);
}

// === KATALOGI ===
// Stwórz katalog (i pośrednie)
Files.createDirectories(Path.of("output/clocks"));

// Listuj pliki w katalogu
try (var stream = Files.list(Path.of("data"))) {
    stream.filter(p -> p.toString().endsWith(".csv"))
        .forEach(p -> System.out.println(p.getFileName()));
}

// === OBSŁUGA BŁĘDÓW ===
```



```
// FileNotFoundException – plik nie istnieje
if (!Files.exists(p1)) {
    throw new FileNotFoundException(p1.toString());
}
// IOException – każdy błąd I/O (checked – trzeba throws lub try-catch)
```

21. Wzorce dla kolokwium — gotowe przepisy

21.1 Klasa abstrakcyjna + podklasy — przepis

```
public abstract class NazwaAbstrakcyjna {
    protected int pole1;          // protected = widoczne w podklasach
    private final String pole2;    // final = niezmiennie po konstruktorze

    protected NazwaAbstrakcyjna(String pole2) {
        this.pole2 = pole2;
    }

    // Metoda abstrakcyjna – podklasy MUSZĄ zaimplementować
    public abstract double oblicz(int x, int y);

    // Metoda zwykła – dziedziczona przez podklasy
    public String getPole2() { return pole2; }

    @Override
    public String toString() {
        return String.format("%s[%s]", getClass().getSimpleName(), pole2);
    }
}

public class PodklasaA extends NazwaAbstrakcyjna {
    public PodklasaA(String pole2) {
        super(pole2);    // MUSI być pierwsze
    }
    @Override
    public double oblicz(int x, int y) { return x + y; }
}
```

21.2 Parsowanie CSV — przepis

```
public class City {
    private final String name;
    private final int timezone;
    private final double latitude;
    private final double longitude;

    private City(String name, int timezone, double latitude, double longitude) {
        this.name = name; this.timezone = timezone;
        this.latitude = latitude; this.longitude = longitude;
    }

    // Parsuje jedną linię: "Warszawa,2,52.2297 N, 21.0122 E"
    private static City parseLine(String line) {
        String[] cols = line.split(",");
        String name = cols[0].trim();
        int tz = Integer.parseInt(cols[1].trim());

        // Parsowanie: "52.2297 N" → +52.2297, "21.0122 W" → -21.0122
        String latStr = cols[2].trim();    // "52.2297 N"
        String[] latParts = latStr.split(" ");
        double lat = Double.parseDouble(latParts[0]);
    }
}
```

```

        if (latParts[1].equals("S")) lat = -lat;

        String lonStr = cols[3].trim(); // "21.0122 E"
        String[] lonParts = lonStr.split(" ");
        double lon = Double.parseDouble(lonParts[0]);
        if (lonParts[1].equals("W")) lon = -lon;

        return new City(name, tz, lat, lon);
    }

    // Wczytuje cały plik, zwraca Map<nazwa, City>
    public static Map<String, City> parseFile(Path path) throws IOException {
        Map<String, City> result = new LinkedHashMap<>();
        List<String> lines = Files.readAllLines(path);
        for (int i = 1; i < lines.size(); i++) { // pomiń nagłówek (i=1)
            City c = parseLine(lines.get(i));
            result.put(c.name, c);
        }
        return result;
    }
}

```

21.3 Własny wyjątek + przechwytywanie — przepis

```

// Checked
class MiastoNieZnalezioneException extends Exception {
    public MiastoNieZnalezioneException(String nazwa) { super(nazwa); }
}

// Rzucanie checked:
public City znajdz(String nazwa) throws MiastoNieZnalezioneException {
    if (!mapa.containsKey(nazwa)) throw new MiastoNieZnalezioneException(nazwa);
    return mapa.get(nazwa);
}

// Przechwytywanie i pomijanie (pętla z try-catch):
for (String nazwa : nazwy) {
    try {
        result.add(znajdz(nazwa));
    } catch (MiastoNieZnalezioneException e) {
        System.out.println(e.getMessage()); // wypisz i idź dalej
    }
}

```

21.4 Sortowanie — przepis

```

// Prosto po liczbie
list.sort(Comparator.comparingDouble(City::getTimeDiff).reversed());

// Przez metodę statyczną (komparator):
public static int worstFit(City a, City b) {
    return Double.compare(Math.abs(b.getDiff()), Math.abs(a.getDiff()));
}
list.sort(City::worstFit); // referencja do metody

```

21.5 Zapis SVG — przepis

```
public String toSvg() {
    StringBuilder sb = new StringBuilder();
    int cx = 200, cy = 200, r = 190;

    sb.append("<svg xmlns=\"http://www.w3.org/2000/svg\" ")
      .append("width=\"400\" height=\"400\">\n");
    sb.append(String.format(
        "<circle cx=\"%d\" cy=\"%d\" r=\"%d\" fill=\"white\" stroke=\"black\" stroke-width=\"4\"/>\n",
        cx, cy, r));

    // Wskazówka – obrót wokół centrum (cx, cy)
    double angle = second * 6.0;    // 6 stopni na sekundę
    sb.append(String.format(
        "<line x1=\"%d\" y1=\"%d\" x2=\"%d\" y2=\"%d\" " +
        "stroke=\"red\" stroke-width=\"2\" " +
        "transform=\"rotate(%.2f %d %d)\"/>\n",
        cx, cy, cx, cy - 150,
        angle, cx, cy));

    sb.append("</svg>");
    return sb.toString();
}
```

Powodzenia na kolokwium! ■